Vamos bien. Ya te dejo una versión ejecutable para enviar a Cuso.

Conclusión ejecutiva: para este tipo de asistente, la arquitectura más sólida hoy no es “entrenar un modelo para memorizar toda la app”, sino combinar: un **overlay en el cliente**, un **orquestador de flujo con estado**, una **capa de conocimiento de la aplicación** y un **LLM con herramientas**. Para conectarlo a sistemas externos o exponer capacidades al agente, **MCP** ya funciona como estándar abierto para pasar contexto y herramientas; para observabilidad, **OpenTelemetry** es el estándar de facto; y para interacción robusta con UI web, conviene usar contratos de elementos estables y locators tipo rol/test-id, como recomiendan las guías de Playwright.

Arquitectura objetivo

1) Capa embebida en la aplicación

Este módulo vive dentro de cada plataforma cliente y es el que hace visible el asistente.

Componentes:

- Launcher del asistente: botón flotante, panel lateral y modo voz/texto.
- Overlay engine: resalta campos, muestra tooltips, checklists, pasos y validaciones.
- UI Context Collector: captura pantalla actual, ruta, componente activo, metadata del formulario, foco, errores visibles y eventos del usuario.
- Action Bridge: recibe instrucciones del backend como “resalta este campo”, “abre este menú”, “muestra hint”, “espera confirmación”.
- Contract Layer: capa de mapeo entre elementos reales del frontend y nombres semánticos estables.

Tecnologías sugeridas:

- Web: React + TypeScript + SDK embebible.
- Overlay: portal DOM propio + popovers/tours.
- Eventos en tiempo real: WebSocket o SSE.
- Contratos UI: `data-testid`, `aria-label`, roles accesibles y nombres canónicos de campos. Playwright recomienda priorizar locators orientados al usuario y contratos explícitos como role/text/test-id, porque son más resilientes.

Regla importante: el asistente no debe depender de scraping frágil del DOM. Debe integrarse por **contratos de UI**.

---

2) API Gateway del asistente

Es la puerta de entrada para todas las plataformas.

Responsabilidades:

- autenticación de tenant, usuario y sesión
- rate limiting
- versionado del protocolo del SDK
- routing hacia orquestación, conocimiento, analítica y acciones

Tecnologías sugeridas:

- FastAPI o NestJS para velocidad de implementación
- Kong / NGINX / API Gateway administrado
- JWT + OAuth2/OIDC

---

3) Orquestador conversacional y de flujo

Este es el cerebro operativo.

Qué hace:

- interpreta la intención del usuario
- decide si responde, guía, pregunta, ejecuta o escala
- mantiene el estado del paso actual
- valida precondiciones
- avanza, retrocede o corrige el flujo

Patrón recomendado:

- **workflow/state machine** para tareas conocidas
- **agentic fallback** solo cuando la tarea es ambigua o abierta

Esto coincide con la distinción actual entre workflows predeterminados y agentes dinámicos; LangGraph, por ejemplo, modela estos procesos con estado persistente y grafos de ejecución.

Decisión de arquitectura:

- Para “registrar cliente”, “crear factura”, “aprobar solicitud”: **FSM / workflow graph**
- Para “explícame qué hace este módulo” o “qué diferencia hay entre cliente local y extranjero”: **LLM + RAG**

---

4) Capa de conocimiento de la aplicación

Aquí vive el “entendimiento” real del sistema.

No debe ser una sola cosa. Debe dividirse en 4 repositorios:

A. App Manifest

Inventario estructurado de la aplicación:

- módulos
- pantallas
- rutas
- formularios
- campos
- validaciones
- dependencias
- permisos
- acciones posibles
- eventos

Formato sugerido:

- JSON/YAML versionado por aplicación y por release

B. Task Graph / Procedure Library

Biblioteca de procedimientos:

- “crear cliente”
- “editar cliente”
- “convertir lead a cliente”
- “emitir nota de crédito”

Cada procedimiento debe incluir:

- objetivo
- precondiciones
- pasos
- campos requeridos
- reglas
- errores comunes
- criterios de éxito
- fallback manual

C. Knowledge Base

Documentación de negocio y ayuda:

- FAQs
- manuales
- definiciones funcionales
- políticas
- explicaciones contextuales

D. Runtime Memory

Memoria de sesión:

- qué intentó hacer el usuario
- en qué paso va
- qué errores encontró
- qué explicación ya vio
- preferencias del usuario

---

5) Capa LLM / AI

Aquí el LLM no “maneja solo la aplicación”. Más bien:

- entiende la intención
- redacta ayudas
- decide qué contexto recuperar
- transforma conocimiento estructurado en guía natural

Recomendación práctica:

- usar un modelo frontier vía API para comprensión y generación
- usar modelos pequeños para clasificación, re-ranking o guardrails si hace falta eficiencia

Patrón recomendado:

- `Intent Classifier`
- `Flow Selector`
- `Retriever`
- `Response Composer`
- `Action Planner`

Muy importante: el conocimiento de campos y funcionalidades debe venir principalmente del **App Manifest + Task Graph + KB + telemetría**, no de fine-tuning pesado.

---

6) Capa RAG y búsqueda

Tu pregunta anterior iba al punto correcto: **el entrenamiento no debería quedar “metido” dentro del modelo como única fuente de verdad**.

Qué va en RAG:

- manuales
- descripciones de módulos
- políticas
- ayudas
- ejemplos
- cambios recientes

Qué no debería depender solo de RAG:

- ubicación exacta de un botón
- nombre exacto de un campo
- orden operativo de un flujo crítico
- permisos y validaciones duras

Eso debe venir del **manifiesto estructurado**.

Arquitectura recomendada:

- Vector DB para semántica
- Índice keyword/metadata para precisión
- Re-ranker
- Context builder

---

7) Action Execution Layer

Si más adelante quieres pasar de “guiar” a “asistir activamente”, necesitas una capa de acciones.

Niveles de autonomía:

1. **Read-only guidance**: solo observa y guía
2. **Assisted action**: preselecciona, abre menú, enfoca campo
3. **Auto-fill with confirmation**: propone completar
4. **Autonomous execution**: ejecuta acciones con aprobación

Tecnologías sugeridas:

- cliente embebido para acciones locales
- Playwright solo para automatización controlada, testing, o entornos compatibles
- event sourcing para auditar cada acción

---

8) Telemetría, observabilidad y aprendizaje

Aquí es donde el sistema mejora de verdad.

OpenTelemetry hoy se usa como framework estándar para generar, recolectar y exportar **traces, metrics y logs**, y su collector permite centralizar el pipeline de observabilidad.

Eventos a capturar:

- user_struggled
- repeated_validation_error
- abandoned_flow

- help_requested
- hint_displayed
- task_completed
- assistant_wrong
- suggestion_accepted

KPIs:

- task success rate
- time-to-complete
- hint acceptance rate
- deflection de soporte
- error recurrence rate
- confidence vs actual success

Arquitectura lógica resumida

```
[SDK embebido / Overlay]
  ├── Chat / Voice UI
  ├── UI Context Collector
  ├── Highlight / Tooltip Engine
  └── Action Bridge
        |
        v
[Assistant Gateway]
        |
        v
[Orchestrator]
  ├── Intent Classifier
  ├── Flow Engine (FSM / Graph)
  ├── Policy & Guardrails
  ├── Action Planner
  └── Response Composer
    |       |       |
    |       |       ├── [RAG / Vector Search]
    |       |       ├── [App Manifest Store]
    |       |       ├── [Task Graph Library]
    |       |       └── [Knowledge Base]
    |
    └── [Action Execution Layer]
        |
        v
[Telemetry / Analytics / OTel]
```

Qué “entrenamiento” sí haría y cuál no

No recomendado como estrategia principal

- Fine-tuning grande para que “se aprenda toda la app”
- Depender de prompts largos con el DOM entero
- Dejar el flujo crítico a un agente 100% libre

Sí recomendado

1. **Modelado estructurado del sistema**
    - generar App Manifest automáticamente desde frontend/backend
    - normalizar campos, rutas, permisos, acciones
2. **Biblioteca de tareas**
    - definir top 20 tareas por plataforma
    - representar cada tarea como workflow declarativo
3. **RAG funcional**
    - indexar ayuda, manuales, políticas y explicaciones
4. **Aprendizaje por telemetría**
    - detectar dónde fallan los usuarios
    - sugerir nuevos walkthroughs
    - ajustar prompts y pasos según evidencia real
5. **Modelos livianos opcionales**
    - clasificador de intención
    - clasificador de “usuario atascado”
    - ranking de próxima mejor ayuda

Diseño para producto multi-tenant / multi-plataforma

Como tú quieres venderlo como módulo reusable, propongo separar en dos partes:

Core platform

Común para todos:

- gateway
- orquestador
- RAG
- observabilidad
- analítica
- administración de tenants
- consola de configuración

Per-app adapter

Específico por cliente/plataforma:

- App Manifest
- mapeo de componentes
- Task Graph
- permisos
- branding del asistente
- conectores a APIs del negocio

Resultado: el 70–80% del producto queda reutilizable; el 20–30% se adapta por plataforma.

Stack tecnológico recomendado

Frontend SDK

- React + TypeScript
- Zustand o Redux Toolkit
- Floating UI / Radix para overlays
- WebSocket/SSE
- i18n desde el inicio

Backend

- Python FastAPI para orquestación AI
- Node/NestJS si quieres SDK/backend homogéneo JS
- Redis para estado efímero y colas ligeras
- Postgres para configuración, tenants, tasks, auditoría
- S3 compatible para assets y snapshots

AI / Search

- LLM por API
- embeddings para KB
- pgvector, Pinecone, Weaviate o Qdrant según escala
- reranker opcional
- LangGraph o motor propio de workflows con estado; LangGraph está orientado precisamente a procesos stateful y workflows/agents.

Integración / estándares

- MCP para exponer herramientas y contexto de forma estándar entre asistentes y sistemas externos. MCP usa JSON-RPC 2.0 y define hosts, clients y servers para compartir contexto y capacidades.

Observabilidad

- OpenTelemetry + Collector
- Prometheus + Grafana
- Loki / ELK
- Sentry para errores frontend

Automatización / validación

- Playwright para pruebas de walkthroughs, smoke tests de contratos UI y entornos controlados. Playwright además dispone de codegen y herramientas de locators útiles para descubrir y estabilizar selectores.

Plan de implementación para Cuso

Fase 0 — Descubrimiento técnico

Duración: 1 semana

Entregables:

- inventario de plataformas objetivo
- top 10 tareas críticas por plataforma
- evaluación de capacidad de integración frontend
- decisión de contrato UI (`data-testid`, roles, labels, ids canónicos)

Preguntas que debe responder Cuso:

- ¿podemos insertar SDK en frontend?
- ¿hay design system común?
- ¿hay APIs para consultar catálogos, reglas y permisos?
- ¿hay entornos staging confiables?

Fase 1 — MVP de guidance

Duración: 2 a 4 semanas

Objetivo: asistente que guía, no automatiza.

Alcance:

- panel lateral
- captura de contexto UI
- overlay con highlight + tooltip

- 3 a 5 flujos críticos
- manifest manual
- KB mínima
- trazabilidad completa

Entregables:

- SDK embebible
- assistant gateway
- flow engine básico
- consola simple de configuración
- telemetría inicial

Definición de éxito:

- completar 3 tareas sin soporte humano
- latencia de respuesta aceptable
- analítica funcionando

Fase 2 — Knowledge + RAG

Duración: 2 a 3 semanas

Objetivo: mejorar comprensión y explicaciones.

Alcance:

- indexar manuales y ayudas
- embeddings + vector search
- respuesta contextual
- grounding por módulo/pantalla/rol

Entregables:

- pipeline de ingesta
- reindexación por release
- política de versionado del conocimiento
- prompts base y guardrails

Fase 3 — App Manifest y Task Graph automáticos

Duración: 3 a 5 semanas

Objetivo: reducir trabajo manual por plataforma.

Alcance:

- generador de manifiesto desde frontend/backend
- parser de formularios
- taxonomía de campos
- DSL de workflows

Entregables:

- esquema estándar de manifiesto
- validador de contratos UI
- librería de procedimientos reutilizable

Fase 4 — Acciones asistidas

Duración: 2 a 4 semanas

Objetivo: pasar de “te explico” a “te ayudo a ejecutar”.

Alcance:

- abrir menús
- enfocar campos
- autocompletar con confirmación
- validación previa de acción
- auditoría detallada

Controles obligatorios:

- permisos por acción
- confirmación humana
- rollback cuando aplique
- logging completo

Fase 5 — Multi-tenant productización

Duración: 3 a 6 semanas

Objetivo: convertirlo en producto reusable.

Alcance:

- tenant isolation
- branding por cliente
- configuración por app

- dashboard SaaS
- billing / usage metering
- plantillas de onboarding por industria

Backlog técnico recomendado

Backend

- definir esquema `app_manifest`
- definir esquema `task_graph`
- servicio `session_state`
- servicio `retrieval`
- servicio `policy_engine`
- servicio `telemetry`
- servicio `tenant_config`

Frontend SDK

- init script embebible
- contexto de ruta/pantalla
- registry de elementos UI
- overlay engine
- handoff de eventos
- soporte accesibilidad

Datos / AI

- pipeline de ingesta KB
- embeddings
- versionado por release
- prompts de guidance
- evaluación de respuestas
- clasificador de intención

DevOps

- CI/CD
- feature flags
- observabilidad OTel
- staging harness
- pruebas Playwright

Riesgos y mitigaciones

Riesgo: cambios frecuentes en el frontend rompen la guía.
Mitigación: contratos UI estables y validación automática por release.

Riesgo: el LLM alucina rutas o campos.
Mitigación: grounding obligatorio con manifest/task graph; si no hay evidencia, no inventa.

Riesgo: demasiada latencia.
Mitigación: separar guidance determinista de generación abierta; cache por pantalla/tarea.

Riesgo: experiencia invasiva.
Mitigación: triggers inteligentes, suppressions, niveles de ayuda.

Riesgo: seguridad y cumplimiento.
Mitigación: redacción de PII, scopes, auditoría, aprobación humana para acciones.

Recomendación final para Cuso

Que construya la **v1 con esta filosofía**:

1. **No entrenar la app dentro del modelo.**
2. **Modelar la app como conocimiento estructurado.**
3. **Usar LLM para entender y explicar, no para adivinar.**
4. **Guiar primero; automatizar después.**
5. **Capturar telemetría desde el día 1.**
6. **Diseñar multi-tenant desde el esquema, aunque el MVP sea single-tenant.**

Si quieres, en el siguiente mensaje te lo convierto en un formato más “ejecutivo” de una página para reenviárselo directo a Cuso.

Muy buena referencia. Cloudflare está construyendo exactamente el tipo de asistente que tú quieres — pero con un enfoque **muy pragmático y productizable**. Te hice un análisis profundo y te saco **los patrones reutilizables clave** para tu arquitectura.

🧠 Qué hace realmente el asistente de Cloudflare

Hay **tres piezas principales** (no es solo un chat bonito):

1. Cloudflare Agent (el más parecido a tu caso)

- Vive dentro del dashboard
- Entiende lenguaje natural
- Puede:
  - responder preguntas
  - ejecutar acciones (crear DNS, reglas, etc.)
  - hacer diagnósticos
  - navegar y configurar el sistema

👉 Esto es EXACTAMENTE el tipo de asistente guiado + operativo que estás diseñando.

---

2. AI Assistant en dashboards (ej: Security Analytics)

- Convierte preguntas en queries complejas
- Interpreta datos y genera visualizaciones automáticamente

👉 Aquí el patrón es:
“lenguaje natural → acción estructurada + UI resultante”

---

3. Cursor / asistentes internos

- Basados en LLM + documentación + contexto
- Especializados por dominio (developers, seguridad, etc.)

🔍 Lo más importante: patrones de diseño que puedes copiar

Aquí viene lo valioso 👇

🧩 1. Arquitectura basada en “capabilities” (NO en prompts)

Cloudflare NO hace esto:

> “LLM sabe todo y decide todo”

Hace esto:

- El LLM **decide qué capability usar**
- El sistema ejecuta esa capability

Ejemplo real:

Usuario:

> “crea un DNS record para mi dominio”

Flujo:

1. LLM detecta intención
2. Mapea a capability: `create_dns_record`
3. Llama API interna
4. Pide confirmación
5. Ejecuta

👉 Esto es clave:
LLM = router / planner
Sistema = ejecutor

🧠 2. Tool-based architecture (muy importante)

El agente de Cloudflare puede:

- llamar APIs
- hacer diagnósticos
- consultar docs
- ejecutar comandos

👉 Esto es exactamente:

> arquitectura basada en herramientas (tools)

Traducción a tu caso:

Tu asistente debe tener tools como:

- `navigate_to_module`
- `highlight_field`
- `validate_input`
- `submit_form`

- `get_field_definition`
- `get_business_rules`

🧭 3. Separación brutal entre conocimiento y ejecución

Cloudflare separa:

Capa	Qué hace
Docs / knowledge	explicar
APIs	ejecutar
Diagnostics	analizar
LLM	orquestar

👉 Esto coincide con lo que te recomendé antes:

- Manifest (estructura)
- Task Graph (flujo)
- RAG (explicación)

🛡️ 4. Confirmación obligatoria para acciones críticas

Cloudflare:

> "Write operations require approval"

👉 Este patrón es CRÍTICO para tu sistema.

Niveles de seguridad:

- leer → automático
- sugerir → automático
- ejecutar → confirmación

⚙️ 5. AI Gateway (control centralizado)

Cloudflare tiene un componente clave:

👉 **AI Gateway**

- logging de prompts
- control de costos
- caching
- rate limiting
- fallback de modelos

Traducción directa a tu arquitectura:

Necesitas un módulo:

👉 **AI Control Layer**

- logs de prompts/respuestas
- métricas
- fallback entre modelos
- control de latencia
- auditoría

🌐 6. Edge + baja latencia

Cloudflare ejecuta AI cerca del usuario (edge GPUs)

👉 Insight:
Tu asistente debe:

- ser rápido (<300ms para acciones UI)
- separar:
  - decisiones rápidas (local/cache)
  - razonamiento pesado (LLM)

🧪 7. “Diagnóstico como feature”

Cloudflare no solo guía.

También:

- hace DNS lookup
- valida certificados

- prueba endpoints

👉 Esto es BRUTALMENTE útil.

Para tu producto:

Agrega tools tipo:

- `validate_form_errors`
- `detect_missing_fields`
- `analyze_user_block`
- `suggest_fix`

🔄 8. Integración directa con APIs (no solo UI)

Cloudflare permite:

> ejecutar acciones directamente vía API

👉 Esto es clave:

Tu sistema no debe depender solo del frontend.

Debe poder:

- leer backend
- ejecutar backend
- validar reglas reales

🔐 9. Seguridad como parte del diseño

Cloudflare integra:

- control de prompts
- prevención de fuga de datos
- governance de uso

👉 Para tu arquitectura:

- masking de datos
- control por roles

- logs auditables

🧱 Arquitectura adaptada (inspirada en Cloudflare)

Aquí te dejo TU versión mejorada 👇

🔷 Núcleo del sistema

```
[Client SDK]
  ├── UI Overlay
  ├── Context Collector
  └── Action Executor
        |
        v
[Assistant Gateway]
        |
        v
[Orchestrator (LLM + Rules)]
  ├── Intent Detection
  ├── Tool Selection
  ├── Flow Engine
  └── Response Generator
        |
        v
[Tool Layer]
  ├── UI Tools
  ├── API Tools
  ├── Diagnostics Tools
  ├── Knowledge Tools
        |
        v
[Knowledge Layer]
  ├── App Manifest
  ├── Task Graph
  ├── Vector DB
  └── Session Memory
        |
        v
[AI Gateway / Observability]
```

🚀 Lo más valioso que debes copiar (prioridad)

Si solo implementas esto, ya tienes algo competitivo:

🔥 TOP 6 cosas de Cloudflare que debes replicar:

1. **Tool-based agent (NO prompt-only)**
2. **Separación: LLM ≠ lógica del sistema**
3. **Confirmación en acciones críticas**
4. **App capabilities como APIs estructuradas**
5. **AI Gateway (observabilidad y control)**
6. **Diagnóstico automático (game changer)**

🧠 Insight clave (muy importante)

Cloudflare NO intenta que el AI “entienda la UI completa”.

Hace esto:

👉 convierte el sistema en:

- APIs
- tools
- capabilities

Y luego:
👉 el AI solo decide **qué usar**

🧭 Recomendación estratégica para ti

Tu producto debería posicionarse como:

> “AI Guidance + AI Execution Layer for SaaS platforms”

No como:

> “chatbot inteligente”

Si quieres siguiente paso

Puedo hacerte algo muy potente:

👉 Diseñar **tu “Agents SDK” estilo Cloudflare**

- cómo exponer tools
- cómo registrar capabilities
- cómo conectar cualquier app en 1 día

O incluso:
👉 definir el **modelo de negocio SaaS** de esto

Solo dime y lo armamos.

Sí. Hice una revisión más profunda y, visto con ojos de arquitecto de software, Cloudflare es una **muy buena inspiración**, pero **no debe copiarse tal cual** para tu caso. La parte valiosa no es “el chat”, sino su combinación de **tools/capabilities**, **estado persistente**, **aprobación humana**, **observabilidad** y **separación clara entre conocimiento y ejecución**. Además, su propio **Cloudflare Agent** está en beta, así que conviene tomarlo como referencia de patrones, no como blueprint literal de producto final.

1) Qué ofrece Cloudflare realmente

Cloudflare Agent, dentro del dashboard, promete cuatro cosas muy concretas: responder preguntas sobre productos y configuraciones, ejecutar operaciones de API con aprobación para acciones de escritura, correr diagnósticos de red como DNS/certificados/conectividad, y buscar documentación relevante. Eso ya te dice mucho sobre su diseño: no es solo un LLM conversando, es un **asistente conectado a herramientas reales del sistema**.

En paralelo, Cloudflare ha ido metiendo asistentes especializados en superficies concretas. En Security Analytics, el asistente traduce lenguaje natural a filtros/consultas y hasta genera series y gráficos; en WAF Rule Builder, ayuda a construir reglas complejas desde lenguaje natural. Ese patrón es importante porque muestra que Cloudflare no usa un único “mega agente”, sino **copilotos por dominio**, cada uno aterrizado en una superficie y un conjunto de acciones bien definido.

2) La arquitectura subyacente que se alcanza a ver

Lo más revelador está en su stack de Agents. Cloudflare documenta que sus agentes corren sobre **Workers + Durable Objects**, donde cada agente es un microservidor con su propio estado, base SQL, WebSockets y scheduling; además soportan herramientas server-side, herramientas client-side en el navegador, flujos de aprobación humana, workflows multi-step, y exposición/consumo de tools vía MCP.

Desde un punto de vista de arquitectura, eso significa que Cloudflare no está apostando por un backend “stateless” clásico para agentes. Está apostando por un modelo **stateful per session/per agent**, con comunicación en tiempo real y capacidad de pausar, reanudar, aprobar, reintentar y seguir una conversación o flujo con memoria persistente. Para un asistente guiado dentro de una aplicación empresarial, esa decisión es técnicamente muy sensata.

3) El patrón más importante: tools primero, LLM después

La pieza central de su framework es el concepto de **tools**. Cloudflare lo define sobre MCP, con descubrimiento de herramientas, validación de parámetros por JSON Schema, manejo estándar de errores y soporte para mantener estado entre invocaciones. Eso no es un detalle menor: obliga a que las capacidades del sistema estén **tipadas, descubiertas y gobernadas**, en vez de dejarlas escondidas en prompts o código ad hoc.

Esta es, críticamente, la lección más importante para tu caso: el asistente no debe “aprenderse el CRM” como si fuera memoria del modelo. Debe operar sobre un catálogo explícito de capacidades: `navigate_to_module`, `open_form`, `highlight_field`, `get_field_help`, `validate_business_rule`, `submit_record`, `run_diagnostic`, etc. El LLM debe decidir **qué capability invocar** y con qué parámetros; la aplicación debe ser la que realmente sepa cómo ejecutar. Esto es consistente con cómo Cloudflare estructura su ecosistema de tools y MCP servers.

4) Qué me gusta mucho de Cloudflare para tu caso

Primero, la **separación entre orquestación y ejecución**. Cloudflare Agent puede sugerir y operar, pero las acciones se canalizan por herramientas y APIs; además, para escritura hay approval flow. Esa separación reduce el riesgo de alucinación operativa.

Segundo, la **mezcla de tools server-side y client-side**. Cloudflare documenta explícitamente client-side tools que corren en el navegador y devuelven resultados al LLM. Eso es especialmente valioso para ti, porque tu producto necesita interactuar con el frontend real: detectar pantalla, resaltar campos, leer validaciones visibles, confirmar foco, abrir un modal, etc. Esa clase de interacción no debe forzarse desde backend; debe vivir en un SDK del lado cliente.

Tercero, el **human in the loop** como primitiva del framework. Cloudflare distingue approvals durables para procesos de larga duración y patrones de confirmación para tool calls sensibles. Para tu caso eso encaja perfecto: leer y explicar puede ser automático; navegar y resaltar, casi automático; guardar, eliminar, aprobar, facturar o cambiar permisos, siempre con confirmación humana.

Cuarto, la **observabilidad nativa del plano AI**. AI Gateway ofrece logging, analytics, cache, rate limiting, retries y model fallback; además registra prompts, respuestas, costo, duración,

tokens y errores. Arquitectónicamente, eso es excelente porque convierte la capa AI en una plataforma observable y controlable, no en una caja negra.

5) Qué no copiaría de Cloudflare, o copiaría con reservas

No copiaría su **dependencia conceptual en MCP como eje interno de todo**. MCP es muy bueno para interoperabilidad entre agentes y servicios externos, y Cloudflare lo empuja fuerte, incluso con auth OAuth 2.1, transporte remoto y servidores gestionados. Pero en tu caso, dentro de una misma aplicación, meter MCP entre el frontend embebido y el core del asistente puede agregar complejidad innecesaria. Yo usaría MCP **en el perímetro**, para integraciones externas y extensibilidad, no necesariamente como protocolo interno principal del producto.

Tampoco copiaría ciegamente el enfoque **agent-per-session sobre infraestructura stateful** si tu prioridad inicial es vender rápido a múltiples plataformas. Cloudflare puede hacerlo porque controla su plataforma de runtime. Tú probablemente necesitas una arquitectura más portable, quizá con Redis/Postgres para estado y eventos, para no casarte con una sola nube ni reescribir los adaptadores por cliente. La idea correcta a copiar es el **estado persistente y reanudable**, no necesariamente Durable Objects como implementación exacta. Esto lo deduzco por sus docs de Agents y Durable Objects; el principio sí es fuerte, la implementación específica es contextual.

Y no extrapolaría demasiado desde sus asistentes de analytics/WAF a tu caso de “guía operativa de formularios”, porque ahí el problema es distinto. En Security Analytics, el reto es traducir lenguaje natural a queries y gráficos; en tu producto, el reto principal es **mapear intención a flujo UI + reglas de negocio + contexto visual**. Hay similitudes, pero no son el mismo tipo de agente.

6) La mejor forma de aplicar Cloudflare a tu caso

Mi recomendación crítica es esta: toma de Cloudflare **el patrón**, no la réplica.

A. Qué sí debes adoptar casi tal cual

Adopta una arquitectura **tool-based**. Define un catálogo formal de capabilities con contratos estrictos, validación de inputs, errores controlados y permisos. Eso está alineado con la filosofía de tools y MCP de Cloudflare.

Adopta **estado de sesión persistente**. El asistente debe saber en qué pantalla está el usuario, qué intentó hacer, qué paso va, qué errores aparecieron, y qué explicaciones ya recibió. Esa necesidad está muy bien resuelta conceptualmente en el stack de Agents.

Adopta **approval flows** para acciones sensibles. Cloudflare lo trata como patrón de primera clase; en software empresarial eso no es opcional, es diseño serio.

Adopta **AI observability** desde el día 1. Necesitarás ver prompts, tool calls, tasa de éxito, fallos por pantalla, costos y latencias. AI Gateway muestra por qué esto debe ser una capa propia.

B. Qué debes adaptar

Donde Cloudflare usa herramientas para operar su propio ecosistema, tú debes construir un **Application Capability Layer** por cada plataforma cliente. Esa capa no debería exponer "DOM crudo"; debería exponer semántica de negocio y UI, por ejemplo:

- `go_to("customers.list")`
- `start_task("customer.create")`
- `highlight("customer.first_name")`
- `explain("customer.type")`
- `validate("customer.create")`
- `submit("customer.create")`

Eso sería tu equivalente al catálogo de tools que Cloudflare conecta por Agent/MCP.

También debes adaptar el patrón de client-side tools a un **SDK embebido universal**. Ese SDK será la pieza más valiosa de tu producto, porque es la que permite navegar, leer contexto visual, señalar campos, escuchar eventos y aplicar overlays. Cloudflare confirma que el split server-side/client-side es una pieza real de su framework; para ti debería ser una decisión fundacional.

C. Qué no debes hacer

No hagas un sistema donde el LLM improvise directamente sobre el DOM. No es robusto, no es portable, y escala mal cuando cambian las pantallas. Cloudflare, en esencia, evita eso encapsulando capacidades y usando tools. Esa es precisamente la parte correcta de su filosofía.

7) Arquitectura recomendada inspirada en Cloudflare, pero mejor ajustada a tu objetivo

Te la dejaría así:

1. Embedded Client SDK

- panel lateral
- overlays
- collector de contexto
- client-side tools
- event stream

2. Assistant Gateway

- auth
- tenant routing
- rate limits
- audit

3. Orchestrator

- intent detection
- flow selection
- tool planner
- response composer
- HITL manager

4. Capability Layer

- navegación UI
- formularios
- validaciones
- acciones de negocio
- diagnósticos

5. Knowledge Layer

- App Manifest
- Task Graphs
- KB/RAG
- Session Memory

6. Observability Layer

- logs AI
- traces de tool calls
- analytics de adopción
- costos, latencia, errores

Esta arquitectura conserva lo mejor de Cloudflare —tools, estado, approvals, observabilidad— pero la aterriza a un producto multi-plataforma de guidance operacional. Cloudflare aporta los patrones base; la diferencia importante es que tú necesitas centrar el diseño en **adaptadores por aplicación** y en un **SDK embebido** que viva dentro de sistemas de terceros.

8) Mi juicio final como arquitecto

Sí, Cloudflare es una referencia muy valiosa.
Pero la enseñanza correcta no es: “hagamos un Cloudflare Agent para CRM”.

La enseñanza correcta es:

- modela capacidades explícitas

- separa guidance, execution y approval
- mantén estado durable de sesión
- usa tools client-side y server-side
- agrega observabilidad de IA como capa formal
- usa MCP para extensibilidad e integraciones, no como dogma interno

Ese enfoque sí lo llevaría a producción.

La parte que más rescataría de Cloudflare para tu caso es esta: **el agente no reemplaza la arquitectura de la aplicación; se monta encima de una arquitectura bien expuesta**. Cuando eso está bien hecho, el asistente puede explicar, guiar, diagnosticar y ejecutar con seguridad. Cuando no, termina siendo un chatbot bonito con respuestas frágiles. Eso es la lectura arquitectónica más importante que deja su stack actual.

Puedo convertir este análisis en un documento ejecutivo con estas tres secciones: **"qué copiar de Cloudflare", "qué no copiar", y "arquitectura recomendada para nuestro producto"**.

Perfecto. Te dejo un **documento ejecutivo listo para enviar a Cuso**: claro, accionable y con criterio arquitectónico sólido.

📄 Documento Ejecutivo

AI Guided Assistant Platform (inspirado en Cloudflare, adaptado a nuestro caso)

🎯 Objetivo

Diseñar e implementar un **asistente inteligente embebido** que permita a usuarios:

- Navegar sistemas complejos sin entrenamiento previo
- Ejecutar tareas paso a paso
- Recibir guía contextual en tiempo real
- Automatizar acciones con control y seguridad

El sistema debe ser:

- **Multi-plataforma**
- **Modular**
- **Escalable**
- **Basado en capacidades (no en prompts)**

🧠 1. Qué copiar de Cloudflare (Patrones correctos)

1.1 Arquitectura basada en *Tools / Capabilities*

El asistente NO ejecuta lógica directamente.

👉 El LLM:

- entiende intención
- selecciona una herramienta

👉 El sistema:

- ejecuta la acción real

Aplicación directa:

Usuario: "crear cliente"
→ Intent: create_customer
→ Tool: start_customer_creation_flow
→ Sistema ejecuta

1.2 Separación estricta de responsabilidades

Capa	Responsabilidad
LLM	comprensión + decisión
Tools	ejecución
Knowledge	contexto
UI SDK	interacción

👉 Evita acoplamiento y alucinaciones.

1.3 Estado persistente (clave crítica)

El asistente debe saber:

- en qué pantalla está el usuario
- qué intenta hacer
- en qué paso va
- qué errores ocurrieron

👉 Cloudflare usa agentes stateful
👉 Nosotros debemos replicar el concepto (no necesariamente la tecnología)

1.4 Human-in-the-loop (no negociable)

Acciones sensibles requieren confirmación:

Tipo acción	Política
Leer	automático
Guiar	automático
Sugerir	automático
Ejecutar	requiere confirmación

1.5 Observabilidad de AI (nivel enterprise)

Debemos registrar:

- prompts
- decisiones del agente
- tools invocadas
- errores
- éxito de tareas
- latencia
- costo

👉 Cloudflare AI Gateway → referencia directa

1.6 Client-side + Server-side tools

👉 Insight clave:

- Backend: lógica, reglas, APIs
- Frontend: UI real, DOM, interacción

Tu sistema NECESITA ambos.

⚠️ 2. Qué NO copiar de Cloudflare

2.1 No usar MCP como núcleo interno

MCP es útil para integraciones externas.

❌ No usarlo como base interna del sistema
✅ Usarlo como capa de extensión

2.2 No depender del DOM como fuente de verdad

❌ scraping dinámico
❌ selectors frágiles

✅ usar:

- contratos UI (`data-testid`)
- App Manifest estructurado

2.3 No intentar que el LLM “se aprenda el sistema”

❌ fine-tuning pesado
❌ prompts gigantes

✅ usar:

- manifest
- task graph
- tools

2.4 No sobre-ingenierizar stateful infra (al inicio)

Cloudflare usa Durable Objects.

👉 Nosotros:

- Redis + DB → suficiente para v1
- luego evolucionar

🏗️ 3. Arquitectura recomendada

🔷 3.1 Vista general

```
[Client SDK]
  ├── UI Overlay
  ├── Context Collector
  ├── Event Listener
  └── Action Executor
        |
        v
[Assistant Gateway]
        |
        v
[Orchestrator]
  ├── Intent Engine
  ├── Flow Engine (FSM)
  ├── Tool Planner
  ├── Response Generator
  └── HITL Manager
        |
        v
[Capability Layer]
  ├── UI Tools
  ├── API Tools
  ├── Validation Tools
  ├── Diagnostic Tools
        |
        v
[Knowledge Layer]
  ├── App Manifest
  ├── Task Graph
  ├── RAG / KB
  └── Session Memory
        |
        v
[Observability Layer]
```

🧩 4. Componentes clave

4.1 Client SDK (core del producto)

Debe permitir:

- overlay visual
- highlight de campos
- navegación guiada
- captura de contexto UI
- ejecución de acciones UI

👉 Este es tu diferenciador principal

4.2 App Manifest (CRÍTICO)

Define la estructura del sistema:

```
{
 "module": "customers",
 "routes": ["/customers", "/customers/create"],
 "fields": [
   {
     "id": "first_name",
     "label": "Nombre",
     "required": true,
     "type": "text"
   }
 ]
}
```

👉 reemplaza dependencia del DOM

4.3 Task Graph (flujos)

```
create_customer:
 step1 → go_to(customers.create)
 step2 → fill(first_name)
 step3 → fill(last_name)
 step4 → validate
```

step5 → submit

👉 convierte procesos en workflows controlados

4.4 Capability Layer (Tools)

Ejemplos:

navigate_to(module)
highlight(field)
explain(field)
validate(form)
submit(form)
detect_errors()

👉 corazón del sistema

4.5 Orchestrator

Decide:

- qué quiere el usuario
- qué flujo usar
- qué tool ejecutar
- cuándo pedir confirmación

4.6 Knowledge Layer

Divide en:

- estructurado → manifest + tasks
- semántico → RAG (docs, FAQs)
- dinámico → memoria de sesión

4.7 Observability

Registrar:

- éxito de tareas

- puntos de fricción
- errores recurrentes
- abandono de flujos

🚀 5. Plan de implementación

🟢 Fase 1 — MVP (3–4 semanas)

Objetivo: guidance funcional

Incluye:

- SDK básico
- overlay + highlight
- 3 flujos (ej: crear cliente)
- manifest manual
- orquestador simple

🟡 Fase 2 — Inteligencia contextual (2–3 semanas)

- RAG (docs)
- mejores respuestas
- explicación de campos
- fallback inteligente

🟠 Fase 3 — Automatización asistida (3–4 semanas)

- ejecutar acciones UI
- validación automática
- confirmación antes de ejecutar

🔵 Fase 4 — Escalabilidad multi-plataforma

- adaptadores por app
- consola de configuración
- tenant management

🟣 Fase 5 — Optimización y producto SaaS

- métricas avanzadas
- personalización
- templates por industria

🧠 6. Principios de diseño (NO negociables)

1. **Capabilities > Prompts**
2. **Estado persistente siempre**
3. **UI no es fuente de verdad**
4. **LLM no ejecuta lógica crítica**
5. **Todo es observable**
6. **Confirmación en acciones sensibles**
7. **Diseño multi-tenant desde inicio**

🧭 7. Posicionamiento del producto

No vender como:

❌ chatbot
❌ asistente conversacional

Vender como:

✅ **AI Execution Layer for Applications**
✅ **In-app AI Guidance Platform**
✅ **AI Copilot for Business Workflows**

🧨 Insight final (clave estratégica)

Cloudflare no ganó valor por tener AI.

Ganó valor porque:

👉 convirtió su plataforma en **capabilities consumibles por AI**

🔥 Tu oportunidad

Tú no estás construyendo un asistente.

Estás construyendo:

> **una capa universal que convierte cualquier software en "AI-operable"**

Si quieres, el siguiente paso es brutal:

👉 te diseño el **SDK (API + contratos + estructura interna)**
eso es lo que realmente convierte esto en producto SaaS escalable.

Perfecto. Vamos al nivel producto.

Te propongo el **SDK del asistente** como si ya fuéramos a construir la primera versión comercial: qué expone, cómo se integra, cómo registra capacidades, cómo conversa con el backend y cómo se adapta a distintas plataformas.

SDK del Asistente Guiado

Objetivo

Crear un **SDK embebible** que permita convertir cualquier aplicación web en una aplicación "AI-guidable" y luego "AI-operable", sin rehacer la aplicación completa.

El SDK debe resolver 5 cosas:

1. Capturar contexto de la UI
2. Mostrar guidance visual
3. Ejecutar acciones controladas
4. Exponer capacidades de la app al asistente
5. Reportar telemetría y estado de sesión

1. Principios del SDK

1.1 El SDK no adivina la app

La app debe exponer:

- rutas
- módulos
- campos
- acciones
- validaciones
- eventos clave

1.2 El SDK no contiene lógica de negocio

La lógica de negocio sigue viviendo en la plataforma host.

1.3 El SDK es una capa de interacción y contrato

Es el puente entre:

- frontend real
- backend del asistente
- modelo de conocimiento

1.4 Debe ser incremental

Debe poder instalarse en tres niveles:

Nivel 1 — Observación

Solo lee contexto y muestra ayuda

Nivel 2 — Guidance

Resalta, explica y guía paso a paso

Nivel 3 — Acción asistida

Puede navegar, enfocar, completar y disparar acciones con confirmación

2. Arquitectura interna del SDK

```
@assistant/sdk
├── Core Runtime
├── Context Collector
├── UI Registry
├── Overlay Engine
├── Action Executor
├── Event Bus
├── Session Sync Client
├── Telemetry Client
└── App Adapter API
```

---

3. Módulos del SDK

3.1 Core Runtime

Es el entrypoint.

Responsabilidades:

- inicialización
- configuración
- autenticación con Assistant Gateway
- ciclo de vida
- manejo de sesión

Ejemplo conceptual

```
AssistantSDK.init({
 appId: "crm-platform",
 tenantId: "tenant-001",
 userId: "user-123",
 sessionToken: "jwt-token",
 environment: "production",
 apiBaseUrl: "https://assistant.company.com",
 mode: "guided"
});
```

3.2 Context Collector

Captura el estado actual de la UI.

Debe recoger:

- ruta actual
- pantalla/módulo actual
- formulario activo
- campo enfocado
- errores visibles
- componentes renderizados
- permisos visibles
- selección actual
- breadcrumbs
- modales abiertos
- tablas y acciones disponibles

Output esperado

```
{
 "route": "/customers/create",
 "screenId": "customers.create",
 "focusedField": "first_name",
 "visibleFields": [
   "first_name",
   "last_name",
   "document_type",
   "customer_type"
 ],
 "visibleActions": [
   "save_customer",
   "cancel"
 ],
 "validationErrors": [],
 "uiState": {
   "modalOpen": false,
   "loading": false
 }
}
```

Regla de diseño

Este contexto debe ser:

- liviano
- estructurado
- semántico

No mandar el DOM entero.

3.3 UI Registry

Es el registro local de elementos UI “nombrados”.

Aquí cada pantalla define:

- campos
- acciones
- contenedores
- tablas
- tabs
- modales

Ejemplo

```
sdk.registerScreen({
 id: "customers.create",
 route: "/customers/create",
 title: "Crear cliente",
 fields: [
   { id: "first_name", label: "Nombre", type: "text", required: true },
   { id: "last_name", label: "Apellido", type: "text", required: true },
   { id: "customer_type", label: "Tipo de cliente", type: "select" }
 ],
 actions: [
   { id: "save_customer", label: "Guardar" },
   { id: "cancel", label: "Cancelar" }
 ]
});
```

Esto es clave porque permite desacoplar el asistente del DOM real.

3.4 Overlay Engine

Muestra la capa visual del guidance.

Debe soportar:

- highlight de campos
- tooltip contextual
- checklist paso a paso
- drawer lateral
- focus ring
- flechas o pointers
- mensajes inline
- estados de espera

- confirmaciones

API interna

```
sdk.overlay.highlight("first_name");
sdk.overlay.showTooltip("first_name", {
 title: "Nombre",
 content: "Ingresa el nombre legal del cliente."
});
sdk.overlay.showChecklist([
 "Abrir módulo clientes",
 "Completar datos básicos",
 "Validar información",
 "Guardar registro"
]);
```

3.5 Action Executor

Permite ejecutar acciones permitidas en el frontend.

Tipos de acción:

- navegar
- enfocar
- abrir dropdown
- seleccionar opción
- escribir valor
- click en acción registrada
- scroll a elemento
- abrir modal

API conceptual

```
sdk.actions.navigate("customers.create");
sdk.actions.focus("first_name");
sdk.actions.setValue("first_name", "Juan");
sdk.actions.click("save_customer");
```

Regla crítica

Nunca debe ejecutar acciones fuera del registry o del capability layer aprobado.

3.6 Event Bus

Sistema interno para publicar y consumir eventos.

Eventos:

- route_changed
- field_focused
- field_changed
- validation_failed
- action_clicked
- modal_opened
- assistant_requested
- guidance_started
- guidance_completed

Ejemplo

```
sdk.events.on("field_focused", (payload) => {
 console.log(payload.fieldId);
});
```

3.7 Session Sync Client

Mantiene la sesión del usuario sincronizada con el backend.

Envía:

- contexto actual
- progreso del flujo
- acciones del usuario
- respuestas del asistente
- estado de ejecución

Recibe:

- siguiente paso
- mensajes
- instrucciones visuales
- tool calls aprobadas
- requests de confirmación

3.8 Telemetry Client

Envía datos para observabilidad.

Eventos mínimos:

- assistant_opened
- intent_detected
- step_started
- step_completed
- step_failed
- action_suggested
- action_confirmed
- flow_abandoned
- help_dismissed

---

4. API pública del SDK

Ahora definimos la API que usaría el equipo integrador.

4.1 Inicialización

AssistantSDK.init(config: AssistantConfig): AssistantInstance

AssistantConfig

```
type AssistantConfig = {
 appId: string;
 tenantId: string;
 userId: string;
 sessionToken: string;
 apiBaseUrl: string;
 mode?: "observe" | "guided" | "assisted";
 locale?: string;
 theme?: "light" | "dark";
 debug?: boolean;
};
```

4.2 Registro de pantallas

sdk.registerScreen(screenDefinition: ScreenDefinition): void
sdk.registerScreens(screens: ScreenDefinition[]): void

ScreenDefinition

```
type ScreenDefinition = {
 id: string;
 route: string;
```

```
 title: string;
 fields?: FieldDefinition[];
 actions?: ActionDefinition[];
 sections?: SectionDefinition[];
 metadata?: Record<string, any>;
};
```

4.3 Registro de tareas

sdk.registerTask(taskDefinition: TaskDefinition): void

TaskDefinition

```
type TaskDefinition = {
 id: string;
 title: string;
 description?: string;
 screenId: string;
 steps: TaskStep[];
 requiredPermissions?: string[];
};
```

TaskStep

```
type TaskStep = {
 id: string;
 type: "navigate" | "focus" | "input" | "select" | "validate" | "submit";
 target?: string;
 instruction: string;
 required?: boolean;
};
```

4.4 Registro de capabilities

sdk.registerCapability(capability: CapabilityDefinition): void

CapabilityDefinition

```
type CapabilityDefinition = {
 id: string;
 description: string;
 kind: "ui" | "api" | "diagnostic" | "knowledge";
 inputSchema: object;
 requiresConfirmation?: boolean;
 handler: (input: any, context: ExecutionContext) => Promise<any>;
};
```

Ejemplo

```
sdk.registerCapability({
 id: "highlight_field",
 description: "Resalta un campo visible en la pantalla actual",
 kind: "ui",
 inputSchema: {
   type: "object",
   properties: {
     fieldId: { type: "string" }
   },
   required: ["fieldId"]
 },
 requiresConfirmation: false,
 handler: async ({ fieldId }) => {
   sdk.overlay.highlight(fieldId);
   return { success: true };
 }
});
```

---

5. Contratos semánticos obligatorios

Aquí está la parte más importante de todo el diseño.

El SDK debe trabajar con **IDs semánticos estables**, no con selectores improvisados.

5.1 Reglas de naming

- `screenId`: `customers.create`
- `fieldId`: `customer.first_name`
- `actionId`: `customer.save`
- `taskId`: `customer.create`
- `sectionId`: `customer.basic_info`

5.2 Ejemplo consistente

screen: customers.create
field: customer.first_name
field: customer.last_name
field: customer.type
action: customer.save
task: customer.create

Esto permite:

- portabilidad
- versionado
- analytics
- prompts más limpios
- menos fragilidad

---

6. Capability Layer: diseño recomendado

La app host debe exponer capacidades agrupadas.

6.1 UI Capabilities

- `navigate_to_screen`
- `highlight_field`
- `focus_field`
- `set_field_value`
- `click_action`
- `open_section`
- `scroll_to_element`

6.2 API Capabilities

- `fetch_customer_types`
- `validate_customer_data`
- `create_customer`
- `search_customer`

6.3 Diagnostic Capabilities

- `detect_blocking_errors`
- `check_required_fields_missing`
- `inspect_permission_denied`
- `explain_validation_error`

6.4 Knowledge Capabilities

- `get_field_help`
- `get_business_rule`
- `get_task_guide`

7. Protocolo entre SDK y backend

El backend no debería recibir HTML suelto, sino mensajes estructurados.

7.1 Context Update

```
{
 "type": "context.update",
 "sessionId": "sess-001",
 "screenId": "customers.create",
 "focusedField": "customer.first_name",
 "visibleFields": ["customer.first_name", "customer.last_name"],
 "visibleActions": ["customer.save"],
 "errors": []
}
```

7.2 User Intent

```
{
 "type": "user.intent",
 "message": "quiero registrar un cliente nuevo"
}
```

7.3 Assistant Instruction

```
{
 "type": "assistant.instruction",
 "instructions": [
   {
     "action": "highlight_field",
     "params": {
       "fieldId": "customer.first_name"
     }
   },
   {
     "action": "show_tooltip",
     "params": {
       "target": "customer.first_name",
       "content": "Ingresa el nombre del cliente."
     }
   }
 ]
}
```

7.4 Confirmation Request

```
{
 "type": "assistant.confirmation_required",
 "actionId": "customer.save",
 "reason": "Esta acción guardará el nuevo cliente en el sistema."
}
```

---

8. Modo de integración por adaptadores

Para que el producto sirva a múltiples plataformas, cada app debe implementar un adaptador.

8.1 Interfaz del adaptador

```
interface AppAdapter {
 getCurrentScreen(): string;
 getVisibleContext(): Promise<UIContext>;
 navigate(screenId: string): Promise<void>;
 highlight(targetId: string): Promise<void>;
 executeAction(actionId: string, params?: any): Promise<any>;
}
```

8.2 Ventaja

El core del SDK no cambia; solo cambia el adaptador.

Eso te da verdadero SaaS multi-plataforma.

---

9. Estructura de paquetes recomendada

```
packages/
├── assistant-sdk-core
├── assistant-sdk-react
├── assistant-sdk-overlay
├── assistant-sdk-telemetry
├── assistant-sdk-types
├── assistant-adapter-crm
├── assistant-adapter-erp
└── assistant-devtools
```

---

10. Seguridad y control

No permitas ejecución libre.

Toda capability debe tener:

- schema de input
- permisos
- clasificación de riesgo
- confirmación si aplica
- auditoría

Ejemplo

type CapabilityRisk = "low" | "medium" | "high";

Política

- low: automático
- medium: automático con log reforzado
- high: confirmación humana obligatoria

---

11. Roadmap de implementación del SDK

Sprint 1

- core runtime
- init config
- event bus
- telemetry client
- registro de pantallas

Sprint 2

- overlay engine
- highlight
- tooltip
- checklist guiado

Sprint 3

- context collector
- session sync
- protocolo backend

Sprint 4

- action executor
- capability registry
- confirmaciones

Sprint 5

- app adapter base
- primer adaptador CRM
- task graph support

Sprint 6

- devtools
- inspector de screens/campos
- validador de contratos

---

12. Qué debe construir Cuso primero

Orden exacto:

Prioridad 1

Assistant SDK Core
Porque sin esto no hay producto integrable.

Prioridad 2

App Adapter Contract
Porque define cómo se conecta cualquier app.

Prioridad 3

Overlay Engine
Porque es lo que el usuario ve.

Prioridad 4

Capability Registry
Porque convierte la app en AI-operable.

Prioridad 5

Protocol + Session Sync
Porque conecta frontend y orquestador.

---

13. Recomendación final de arquitectura

Mi recomendación concreta es esta:

- **frontend:** React + TypeScript
- **SDK:** paquete modular NPM
- **contratos:** TypeScript types + JSON Schema
- **estado local:** Zustand o equivalente
- **tiempo real:** WebSocket
- **telemetría:** OpenTelemetry + eventos propios
- **backend assistant:** FastAPI o NestJS
- **knowledge layer:** App Manifest + Task Graph + RAG
- **acción:** capability-based, nunca DOM libre

---

14. Insight estratégico final

El verdadero producto no es el chat.

El verdadero producto es este triángulo:

- **SDK embebible**
- **Capability model**
- **Orquestación AI**

Si eso queda bien diseñado, luego puedes vender:

- guidance
- copiloto operativo
- automatización asistida
- soporte contextual
- onboarding inteligente

sobre la misma base.

Si quieres, el siguiente paso natural es que te arme el **backend complementario del SDK**: Assistant Gateway, Orchestrator, Capability Router, Session Store y esquema de base de datos.

Sí, y aquí está el punto arquitectónico clave: **esto no se debe manejar "reentrenando el modelo" cada vez que cambia la aplicación**. En operación, los cambios de interfaz, flujos, outputs y reglas de negocio deben tratarse como **cambios versionados de producto**: contratos de UI, manifiestos de pantallas, task graphs, reglas y knowledge packs. El LLM queda como capa de comprensión y orquestación; la verdad operativa vive fuera del modelo. Además, para interacción robusta con UI, conviene apoyarse en contratos estables como roles, labels y `data-testid`, porque son más resilientes que selectores frágiles.

1) Cómo manejar cambios en un ambiente operativo

Yo lo dividiría en **4 tipos de cambio**, porque no todos se tratan igual.

A. Cambios de interfaz

Ejemplos:

- cambió el nombre visual de un botón
- el campo pasó de una pestaña a otra
- cambió el layout
- el selector DOM dejó de existir

Cómo se maneja:

- Mantener un **UI Contract Layer** con IDs semánticos estables (`customer.first_name`, `customer.save`).
- Mapear esos IDs a los elementos reales del frontend.
- Versionar ese contrato por release.
- Correr validación automática por cada despliegue.

Aquí la mejor práctica es no depender del DOM "tal como venga". Playwright recomienda priorizar locators orientados al usuario y contratos explícitos como roles y test ids para que la automatización sea más resistente a cambios.

En operación:

- release entra a staging
- se ejecuta suite de validación de contratos UI
- si falla un mapeo, se marca como "guidance degraded" para esa pantalla
- el sistema no ejecuta acciones automáticas en pantallas no validadas

---

B. Cambios de flujo de trabajo

Ejemplos:

- crear cliente ahora requiere un paso adicional
- el proceso cambió de 4 pasos a 6
- hay un nuevo desvío según tipo de cliente

Cómo se maneja:

- Los flujos deben vivir como **Task Graphs / workflows declarativos**
- Cada flujo debe tener versión
- El orquestador selecciona la versión correcta según release, tenant o feature flag

Regla de oro:
Los flujos no deben estar “hardcodeados” en prompts. Deben estar declarados como assets operativos.

C. Cambios de outputs

Ejemplos:

- cambió el formato de respuesta de una API
- el sistema ahora devuelve nuevos estados
- cambió el payload de validación

Cómo se maneja:

- Adaptadores por aplicación
- Schema validation
- Versionado de contratos API
- Compatibilidad hacia atrás cuando se pueda

Esto también aplica muy bien al patrón de tools: MCP define tools con nombres únicos y esquemas explícitos; esa disciplina de contrato es la correcta para capabilities internas también, aunque no uses MCP como núcleo interno.

D. Cambios de reglas de negocio

Ejemplos:

- ahora clientes extranjeros requieren documento adicional
- cambió la lógica de validación de crédito
- cambió una política de aprobación

Cómo se maneja:

- Separar reglas en un **Business Rules Layer**
- Versionar reglas
- Asociar reglas a:
    - tenant
    - producto
    - país
    - fecha efectiva
- El asistente consulta reglas, no las inventa

Muy importante:
las reglas de negocio son la parte que menos debería depender del LLM. El LLM puede explicarlas, pero no debe ser la fuente de verdad.

---

2) Qué arquitectura operativa recomiendo para manejar estos cambios

Yo pondría un pipeline así:

```
Cambio en app / backend
  ->
Detección de cambio
  ->
Actualización de assets operativos
  - UI Contract
  - Screen Manifest
  - Task Graph
  - Business Rules
  - Knowledge Pack
  ->
Validación automática
  - UI checks
  - task replay
  - contract tests
  ->
Publicación versionada
  ->
Canary release
  ->
Monitoreo
  ->
Promoción a producción
```

Assets que deben versionarse

No solo el código.

También:

- `screen_manifest`
- `task_graph`
- `capability_schema`
- `business_rules`
- `knowledge_pack`
- `prompt_pack`
- `telemetry_schema`

OpenTelemetry es útil aquí porque estandariza generación, exportación y colección de traces, métricas y logs, y además su comunidad está trabajando activamente en convenciones para observabilidad de agentes/GenAI.

---

3) Qué hacer cuando hay un cambio no previsto en producción

Tu sistema debe tener **modos degradados**, no fallar de forma binaria.

Modo normal

- guía
- resalta
- ejecuta acciones permitidas

Modo degradado UI

- puede responder y explicar
- no ejecuta acciones automáticas
- no resalta elementos no validados
- pide al usuario navegación manual

Modo degradado knowledge

- guía solo con flujos validados
- no responde preguntas abiertas sin grounding suficiente

Modo seguro

- solo lectura
- sin ejecución
- sin autocompletado

Eso es mucho más maduro que intentar “aguantar” con el modelo inventando.

---

4) Qué tan útil sería un módulo de aprendizaje por observación de un usuario experto

Muy útil, pero no como reemplazo del modelado explícito.
Lo usaría como acelerador, no como fuente principal de verdad.

Sirve mucho para:

- descubrir flujos reales
- identificar variantes del proceso
- detectar shortcuts que usan expertos
- capturar lenguaje natural real del negocio
- encontrar puntos de fricción frecuentes
- inferir hints útiles y validaciones tácitas

No sirve por sí solo para:

- definir reglas de negocio oficiales
- garantizar cumplimiento
- distinguir workaround de proceso correcto
- manejar edge cases regulatorios

Entonces mi postura arquitectónica es:

Sí al aprendizaje por demostración.
No a depender solo de él.

Whatfix y otras plataformas de adopción digital se enfocan justamente en guiar dentro del flujo de trabajo y medir adopción, engagement y gaps, lo que valida mucho este enfoque de capturar uso real para mejorar la guía.

---

5) Cómo debería funcionar ese módulo de aprendizaje

Yo lo diseñaría en 3 niveles.

Nivel 1 — Recording estructurado

No grabar video bruto como fuente principal.

Capturar eventos estructurados:

- pantalla
- campo enfocado
- valor tipo, no necesariamente el valor real
- validaciones
- secuencia de acciones
- tiempos
- errores y correcciones
- decisiones tomadas

Ejemplo:

```
{
 "screenId": "customers.create",
 "event": "field_changed",
 "fieldId": "customer.type",
 "valueType": "enum",
 "timestamp": "..."
}
```

Nivel 2 — Mining de tareas

A partir de múltiples ejecuciones de expertos:

- detectar secuencias frecuentes
- agrupar variantes
- proponer un task graph candidato
- detectar pasos opcionales vs obligatorios

Nivel 3 — Human curation

Un analista funcional o product owner valida:

- si ese flujo es oficial
- si hay pasos indebidos
- si faltan reglas
- si la explicación es correcta

Nunca publicaría un flujo aprendido automáticamente sin revisión humana para procesos críticos.

---

6) Proceso recomendado de carga inicial de conocimiento

Yo haría la carga inicial en este orden:

Paso 1 — Knowledge base estructural

Extraer del sistema:

- módulos
- rutas
- pantallas
- formularios
- campos
- acciones
- permisos
- catálogos
- mensajes de validación

Esto forma el **App Manifest inicial**.

Paso 2 — Rules extraction

Traer desde backend o documentación:

- reglas de negocio
- validaciones
- dependencias entre campos
- permisos por rol

Esto forma el **Rules Pack**.

Paso 3 — Task seeding

Definir manualmente los 10–20 procesos más críticos:

- crear cliente
- editar cliente
- buscar cliente
- aprobar solicitud
- emitir factura

Esto forma la **Procedure Library**.

Paso 4 — Content/RAG ingestion

Indexar:

- manuales
- FAQs
- SOPs
- documentación interna
- mensajes de ayuda

Esto forma el **Knowledge Pack**.

Paso 5 — Expert demonstration capture

Pedir a usuarios expertos ejecutar los flujos principales y capturar:

- secuencia real
- variantes
- lenguaje natural
- errores comunes

Esto sirve para:

- enriquecer task graphs
- mejorar hints
- entrenar clasificadores ligeros
- generar ejemplos few-shot

Paso 6 — Validation before production

Antes de liberar:

- replay de tareas
- contract tests
- revisión funcional
- pruebas de seguridad
- pruebas de observabilidad

---

7) Cómo deberían hacerse las futuras actualizaciones

Yo haría un proceso tipo CI/CD de conocimiento.

Cada release debería disparar:

1. **Detección de delta**
    - qué pantallas cambiaron
    - qué campos cambiaron
    - qué reglas cambiaron
    - qué outputs cambiaron
2. **Regeneración parcial**
    - screen manifests afectados
    - mappings UI
    - task graphs relacionados
    - embeddings o índices del KB
3. **Pruebas automáticas**
    - UI contracts

    - replay de tareas críticas
    - validación de capabilities
    - smoke tests de guidance
4. **Canary rollout**
    - 5% de usuarios internos o sandbox
    - comparar task success rate
    - medir errores de guidance
5. **Promoción**
    - si KPIs y validaciones pasan

---

8) Qué “aprendizaje” sí haría y cuál no

Sí haría

- aprendizaje supervisado para intención
- mining de secuencias de expertos
- ranking de mejor siguiente paso
- detección de usuarios atascados
- sugerencia de nuevos hints y walkthroughs

No haría como eje principal

- fine-tuning continuo del LLM por cada cambio de interfaz
- aprendizaje online directo en producción sin validación
- que el modelo edite por sí solo reglas o flujos oficiales

---

9) Mi recomendación final

La manera correcta de operarlo es esta:

La app cambia → se actualizan assets operativos → se validan → se despliegan.
No: **la app cambia → esperamos que el modelo se adapte solo**.

Y sobre el aprendizaje por usuario experto:

sí vale mucho la pena, pero como mecanismo de:

- descubrimiento
- aceleración
- mejora continua
- generación de candidatos

no como única fuente de verdad del sistema.

Resumen ejecutivo

- **Cambios de UI**: manejar con contratos semánticos y validación automática.
- **Cambios de flujo**: task graphs versionados.
- **Cambios de outputs**: schemas y adapters versionados, estilo tools con contratos explícitos.
- **Cambios de reglas**: rules engine / rules pack separado del LLM.
- **Aprendizaje por experto**: sí, pero con curación humana.
- **Carga inicial**: manifest + rules + tasks + KB + demostraciones expertas.
- **Actualizaciones futuras**: pipeline CI/CD de conocimiento + canary + observabilidad.

Puedo convertir esto en un **playbook operativo** con roles, procesos, ambientes, artefactos y responsables para que Cuso lo implemente como modelo de operación.

📘 PLAYBOOK OPERATIVO

AI Guided Assistant Platform

🎯 Objetivo del Playbook

Definir **cómo operar, mantener y evolucionar** el asistente en producción cuando:

- cambian interfaces
- cambian flujos
- cambian reglas
- cambian APIs
- cambian necesidades del negocio

🧭 1. Modelo Operativo General

El sistema NO se mantiene reentrenando modelos.

Se mantiene mediante:

Cambios en sistema
→
Actualización de assets
→

Validación automática
→
Despliegue controlado
→
Monitoreo
→
Mejora continua

🧱 2. Assets Operativos (FUENTE DE VERDAD)

Estos son los componentes que realmente gobiernan el asistente:

2.1 App Manifest

Define:

- pantallas
- rutas
- campos
- acciones
- estructura UI

2.2 Task Graphs

Define:

- flujos paso a paso
- dependencias
- caminos alternos

2.3 Capability Registry

Define:

- qué puede hacer el asistente
- cómo ejecutar acciones
- contratos de input/output

2.4 Business Rules Pack

Define:

- validaciones
- condiciones
- reglas de negocio

2.5 Knowledge Pack

Define:

- explicaciones
- FAQs
- manuales
- hints

2.6 Prompt Pack

Define:

- comportamiento del asistente
- estilo
- instrucciones base

2.7 Telemetry Schema

Define:

- qué medir
- cómo medir
- KPIs

🔁 3. Pipeline de Cambios (Core del sistema)

Flujo estándar

Cambio detectado
→
Clasificación del cambio
→
Actualización de assets
→
Validación automática
→
Revisión humana (si aplica)
→
Deploy a staging
→
Canary release
→
Producción

🧩 4. Clasificación de Cambios

Tipo 1 — UI Change

Ej:

- campo cambia de lugar
- botón cambia nombre

Acción:

- actualizar UI mapping
- validar contract
- no afecta lógica

Tipo 2 — Flow Change

Ej:

- nuevo paso en proceso

Acción:

- actualizar Task Graph
- revalidar flujo

Tipo 3 — Rule Change

Ej:

- nueva validación

Acción:

- actualizar Rules Pack
- actualizar validadores

Tipo 4 — API Change

Ej:

- cambia response

Acción:

- actualizar adapter
- validar schemas

Tipo 5 — Knowledge Change

Ej:

- nuevo procedimiento

Acción:

- actualizar Knowledge Pack
- reindexar RAG

🧪 5. Validación Automática

5.1 UI Contract Tests

Validar:

- existencia de campos
- acciones accesibles
- rutas válidas

5.2 Task Replay Tests

Simular:

- ejecución de flujos críticos

5.3 Capability Tests

Validar:

- input/output schemas
- errores controlados

5.4 Regression Tests

Validar:

- que nada existente se rompió

5.5 AI Behavior Tests

Validar:

- intención → acción correcta
- respuesta contextual correcta

🚀 6. Estrategia de Deploy

6.1 Entornos

Entorno	Uso
Dev	desarrollo
Staging	validación
Canary	pruebas reales
Prod	producción

6.2 Canary Release

- 5–10% usuarios
- monitorear:
  - errores
  - task success rate
  - latencia
  - abandono

6.3 Rollback automático

Triggers:

- caída en success rate
- aumento de errores
- latencia crítica

📊 7. Observabilidad (OBLIGATORIO)

Métricas clave

Performance

- latencia

- tiempo de respuesta
- tiempo por tarea

Uso

- sesiones
- intents
- tasks iniciadas

Éxito

- task completion rate
- pasos completados

Fricción

- errores repetidos
- abandono
- solicitudes de ayuda

Calidad AI

- respuestas incorrectas
- fallback rate
- hallucination signals

🧠 8. Módulo de Aprendizaje (Learning System)

8.1 Objetivo

Mejorar el sistema mediante:

- comportamiento real de usuarios

- ejecución de expertos

8.2 Tipos de aprendizaje

A. Observación pasiva

Captura:

- eventos
- errores
- rutas

B. Demostración experta

Usuarios expertos ejecutan flujos

C. Mining automático

Detecta:

- patrones
- variantes
- shortcuts

D. Curación humana

Valida:

- si el flujo es correcto
- si es óptimo
- si cumple reglas

8.3 Pipeline de aprendizaje

Captura eventos
→
Normalización
→

Clustering de flujos
→
Generación de candidatos
→
Revisión humana
→
Conversión a Task Graph
→
Deploy

8.4 Qué NO hacer

- ❌ aprendizaje automático sin validación
- ❌ permitir que el modelo cambie reglas
- ❌ confiar solo en comportamiento observado

📥 9. Carga Inicial de Conocimiento

Paso 1 — App Discovery

Extraer:

- pantallas
- rutas
- campos
- acciones

Paso 2 — Rules Extraction

- validaciones
- dependencias
- permisos

Paso 3 — Task Definition

Definir manualmente:

- top 20 procesos críticos

Paso 4 — Knowledge Ingestion

- documentación
- FAQs
- manuales

Paso 5 — Expert Sessions

- grabar ejecución de expertos
- capturar variantes

Paso 6 — Validación

- replay de flujos
- pruebas de contrato
- QA funcional

🔄 10. Actualizaciones Futuras

Proceso estándar

Nuevo release app
→
Detectar cambios
→
Actualizar assets
→
Ejecutar tests
→
Deploy staging
→
Canary
→
Producción

Automatización recomendada

- diff de manifests
- alertas de cambios UI
- regeneración de embeddings
- test automático de flujos

🧑‍💻 11. Roles y Responsabilidades

🧠 AI Architect

- define arquitectura
- define capability model

🧩 Platform Engineer

- implementa SDK
- integra apps

🧪 QA Automation

- valida flujos
- ejecuta tests

📊 Data Engineer

- telemetría
- pipelines de aprendizaje

📘 Product / Ops

- define procesos

- valida flujos

👩‍🏫 Domain Expert

- ejecuta demostraciones
- valida conocimiento

⚠️ 12. Modos de Falla y Respuesta

UI rota

→ desactivar acciones automáticas
→ fallback a guidance

Rules inconsistentes

→ bloquear ejecución
→ mostrar explicación

LLM falla

→ fallback a:

- flujos predefinidos
- respuestas estáticas

Context incompleto

→ pedir input al usuario
→ reducir automatización

🧨 13. Principios Críticos

1. **Nunca confiar solo en el LLM**
2. **Todo debe ser versionado**
3. **Todo debe ser testeable**
4. **Toda acción debe ser auditable**
5. **La UI no es fuente de verdad**
6. **El usuario siempre tiene control final**
7. **El sistema debe degradar, no fallar**

🧭 14. Roadmap Operativo (Recomendado)

Mes 1

- SDK base
- manifest inicial
- 3 flujos

Mes 2

- capability layer
- RAG básico
- telemetría

Mes 3

- aprendizaje por observación
- automatización parcial

Mes 4+

- multi-tenant

- optimización
- analytics avanzado

🎯 Conclusión Ejecutiva

Este sistema se mantiene así:

👉 **No entrenando modelos constantemente**
👉 **Sino gestionando conocimiento estructurado versionado**

💡 Insight final

Si haces esto bien, no solo construyes un asistente.

Construyes:

> 🧠 **una capa operativa de inteligencia sobre cualquier software**